

7.1 训练概述与算法对比

概述

LeRobot 支持多种主流机器人学习算法，本文对各算法进行横向对比，帮助你选择适合自己任务和硬件条件的训练方案。

算法分类体系

Code block

```
1  LeRobot 支持的算法
2  |—— 模仿学习 (Imitation Learning)
3  |    |—— ACT (Action Chunking with Transformers)
4  |    |—— Diffusion Policy (扩散策略)
5  |    |—— VQ-BeT (矢量量化行为变换器)
6  |
7  |—— 强化学习 (Reinforcement Learning)
8  |    |—— HIL-SERL (人在回路强化学习)
9  |    |—— TDMPCL (基于模型的规划)
10 |
11 |—— 视觉-语言-动作模型 (VLA)
12     |—— SmoLVLA (轻量级, 进阶推荐)
13     |—— Pi0 (效果优秀)
14     |—— Pi0.5 (Pi0 升级版)
15     |—— GR00T N1.5 (NVIDIA, Jetson 优化)
16     |—— WALL-OSS (国产开源)
```

算法横向对比

算法	入门难度	GPU 显存需求	训练时长	推理效果	推荐场景
ACT	★☆☆ 简单	≥8GB	2-8小时	良好	入门首选，精细操作
Diffusion	★★☆ 中等	≥8GB	4-12小时	良好	多模态任务
SmolVLA	★★☆ 中等	≥16GB	4-12小时	优秀	进阶推荐，语言条件
Pi0	★★★ 较难	≥24GB	8-24小时	最优	高精度任务
Pi0.5	★★★ 较难	≥24GB	8-24小时	最优	Pi0 改进版
GR00T N1.5	★★★ 较难	专用Jetson	较长	优秀	Jetson嵌入式部署
WALL-OSS	★★☆ 中等	≥16GB	4-12小时	优秀	国产开源，中文支持

各算法简介

ACT（Action Chunking with Transformers）

- 论文：[Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware]
(<https://arxiv.org/abs/2304.13705>)
- 核心思路：将一段时间内的动作序列（Action Chunk）作为整体预测单元，减少累积误差
- 适合任务：单次抓取、精确放置、固定流程操作
- 入门推荐指数：★★★★★

Diffusion Policy（扩散策略）

- 论文：[Diffusion Policy: Visuomotor Policy Learning via Action Diffusion] (<https://arxiv.org/abs/2303.04137>)
- 核心思路：将动作生成建模为去噪扩散过程，能处理多峰分布的动作
- 适合任务：有多种合理解法的任务（如绕行障碍）

SmolVLA

- 核心思路：轻量级视觉-语言-动作模型，语言条件控制
- 优势：结合视觉理解和自然语言指令，泛化能力强
- 适合任务：需要语言描述的灵活任务

Pi0 / Pi0.5

- 论文：[π 0: A Vision-Language-Action Flow Model] (<https://www.physicalintelligence.company/blog/pi0>)
- 核心思路：基于 Flow Matching 的 VLA 模型，效果最优
- 局限：需要大显存（ $\geq 24\text{GB}$ ），训练时间长

GR00T N1.5

- 来源：NVIDIA 开源
- 核心优势：专为 Jetson 平台优化，可部署到 Jetson AGX Thor 等边缘设备
- 适合场景：需要在嵌入式设备上部署的应用

WALL-OSS

- **来源**：国内开源项目
- **特点**：国产开源，对中文任务描述友好
- **适合场景**：中文场景任务，国内用户使用

硬件要求参考

训练用 GPU 推荐

算法	最低要求	推荐配置
ACT	RTX 3080 (10GB)	RTX 3090/4090 (24GB)
Diffusion	RTX 3080 (10GB)	RTX 3090/4090
SmolVLA	RTX 3090 (24GB)	A100 (40GB)
Pi0 / Pi0.5	RTX 3090 (24GB)	A100/H100
GR00T N1.5	Jetson AGX Thor 专用	Jetson AGX Thor

> **注意**：不建议使用 V100 GPU，该架构（Volta）不支持部分训练依赖。推荐使用 Ampere 架构及以上（RTX 30xx/40xx, A100, H100）。

Mac（Apple Silicon）本地训练

LeRobot 支持 PyTorch **MPS 后端**，Apple Silicon Mac 可直接本地训练，无需 NVIDIA GPU。

算法	16GB 统一内存	24GB 统一内存	32GB 统一内存	64GB+ 统一内存
ACT	✓	✓	✓	✓
Diffusion	✓	✓	✓	✓
SmolVLA	⚠ 吃紧	✓	✓	✓
Pi0 / Pi0.5	✗	⚠ 可尝试	✓	✓
GR00T N1.5	✗	✗	✗	✗ 仅限 Jetson

✓ 可正常训练
 ⚠ 可尝试但可能较慢或内存吃紧
 ✗ 不支持

芯片适用说明：

- **M1 / M2 / M3 / M4 / M5**（基础版，16GB）：ACT 和 Diffusion 可正常训练，是验证完整训练流程的推荐选择
- **M1 Pro/Max · M2 Pro/Max · M3 Pro/Max · M4 Pro/Max · M5 Pro/Max**（24-48GB）：SmolVLA 可流畅训练
- **M1 Ultra · M2 Ultra · M3 Ultra · M4 Ultra · M5 Ultra**（64-192GB）：可训练几乎所有算法

速度参考（相对 RTX 3090）：

芯片	相对训练速度	ACT 预估训练时间
M1/M2（16GB）	约 1/8	16-40 小时
M3/M4/M5（16-24GB）	约 1/5	10-25 小时
M-Max/Ultra 高内存	约 1/3	6-15 小时
RTX 3090（参照基准）	1x	2-8 小时

> **建议：**在 Mac 上用 ACT 跑通完整训练流程（数据采集→训练→推理），正式大规模训练推荐使用云 GPU，详见 [6-云GPU训练环境配置.md](./6-云GPU训练环境配置.md)。

训练监控：wandb 实时曲线

所有算法训练时均支持 wandb 实时监控：

Code block

```
1  # 安装 wandb
2  pip install wandb
3
4  # 登录
5  wandb login
6
7  # 训练时自动上传曲线（在训练命令中添加）
8  --training.use_wandb=true
```

在 wandb.ai 上可以实时查看训练损失曲线，判断训练是否收敛。

训练结果存储

训练完成后，模型权重保存在：

Code block

```
1  # 默认保存路径
2  outputs/train/YYYY-MM-DD_HH-MM-SS_<policy_name>/
3  |—— checkpoints/
```

```
4 | | └─ last/
5 | |   └─ pretrained_model/      # 最终模型（用于推理）
6 | |     └─ training_state/      # 训练状态（用于续训）
7 | |       └─ 010000/            # 步骤 10000 的检查点
8 | |         └─ 020000/
9 | └─ logs/
10 └─ train/                      # 训练日志（tensorboard格式）
```

选择建议

Code block

- | | |
|---------------|---------------------------|
| 1 新手？ | → ACT（最简单，效果可靠） |
| 2 需要语言控制？ | → SmolVLA |
| 3 追求最佳效果？ | → Pi0 / Pi0.5（需要大显存） |
| 4 部署到 Jetson？ | → GR00T N1.5 |
| 5 国内用户/中文任务？ | → WALL-OSS 或 SmolVLA |
| 6 只有小显存？ | → ACT 或 Diffusion（8GB 可用） |

参考资料

- [LeRobot GitHub（算法列表）](#)
- [HuggingFace LeRobot 文档](#)